National University of Sciences & Technology

School of Electrical Engineering & Computer Science (SEECS)

EE-222: Microprocessor Systems

Engineering Semester Project (ESP) Report

Fire & Smoke Detection System

An ATmega328P-Based Real-Time Hazard Monitoring System

Faculty Sponsor	Dr. Moazzam Ali
Lab Engineer	Engr. Muhammad Adnan
Department	Electrical Engineering, SEECS
Semester	Spring 2026

Submitted By

Team Member	Registration No.
Abuzar Shafaat	510694
Laveeza Fatima	515521
Mohammed Imaan Khan	532863
Muhammad Soban Zamir	501214

TABLE OF CONTENTS

Abstract 3
1. Problem Definition and Analysis 3
1.1 The Complex Engineering Problem (CEP) 3
1.2 Why This Problem Matters 4
1.3 Constraints 4
1.4 Engineering Complexity 4
2. Literature Review & Theoretical Foundation 5
2.1 Embedded Systems 5
2.2 Microcontrollers vs. Microprocessors 5
2.3 Why ATmega328P? 5
2.4 Relevant Concepts 6
2.4.1 GPIO (General-Purpose Input/Output) 6
2.4.2 ADC (Analog-to-Digital Converter) 6
2.4.3 Timers and Delays 6
2.4.4 Interrupts 6
2.4.5 I^2C / TWI Communication 6
3. Design and Methodology 7
3.1 Block Diagram 7
3.2 Components Used & Justification 8
3.3 Why ATmega328P — Detailed Justification 8
3.4 Programming Approach (Assembly & C) 9
4. Code Snippet 10
4.1 ADC Read Function 10
4.2 Hazard Decision and Buzzer Control 10
4.3 Pin Direction Configuration 10
5. Flowchart (Logic Explanation) 11
6. Simulation, Modeling, and Implementation 12
6.1 Software Tools Used 12
6.2 Circuit Simulation Results 12
6.3 Transition from Simulation to Hardware 13
7. Results, Testing, and Validation 14
7.1 Headline Metrics 14
7.2 Test Cases — Expected vs. Actual 14
7.3 Accuracy and Reliability 14

8. Technical Analysis (Mathematics + Hardware Link) 15
8.1 Voltage and Current — Ohm's Law 15
8.2 ADC Resolution — MQ-2 Reading 15
8.3 Timing — CPU and Polling Interval 15
8.4 I²C Bit-Rate — TWBR Calculation 15
8.5 Linking the Mathematics to the Circuit 16
9. Innovation, Integration, and Complexity 16
9.1 What Makes This Project Unique 16
9.2 Integration of Hardware and Software 16
9.3 Complexity Level 16
10. Teamwork and Indivi 17
dual Contribution 17
11. Professionalism and Ethical Considerations 17
11.1 Safety 17
11.2 Reliability 18
11.3 Ethical Use of Technology 18
12. Learning Outcomes 18
12.1 AVR Register-Level Programming 18
12.2 I²C Communication and Peripheral Interfacing 18
12.3 Hardware–Software Integration and Debugging 18
13. Conclusion 19
13.1 Future Work 19
14. References 20
Datasheets 20
Books 20
Research and Application Notes 20

Abstract

This report presents the design, implementation, and validation of a Fire & Smoke Detection System developed as the Engineering Semester Project (ESP) for EE-222 Microprocessor Systems. The system addresses the persistent problem of delayed fire detection in homes and small offices, where commercial alarm systems are often prohibitively expensive. The proposed solution is a low-cost, real-time embedded system built around the ATmega328P microcontroller, programmed entirely at the AVR register level using C and the Atmel Studio toolchain.

The design integrates two complementary sensors an MQ-2 gas/smoke sensor read through the on-chip 10-bit ADC, and an infrared flame sensor read through a digital GPIO pin to perform sensor fusion for reliable hazard detection. When either the smoke level exceeds a calibrated threshold (ADC value of 500, equivalent to approximately 2.44 V) or the flame sensor is triggered, the firmware activates a buzzer alarm and updates a 16×2 I^2C LCD with the current status. The complete prototype was simulated in Proteus 8 Professional and subsequently realised in hardware for under PKR 3,000.

Validation across six test cases produced a 100% pass rate, with an end-to-end response time of under 500 ms and a detection accuracy of approximately 95%. The project demonstrates competence in low-level peripheral programming (DDRB, ADMUX, ADCSRA, TWBR, PORTB), I^2C protocol implementation, polling-based real-time control, and the integration of analog and digital sensors with a microcontroller.

1. Problem Definition and Analysis

1.1 The Complex Engineering Problem (CEP)

Fire-related incidents continue to cause significant loss of life and property worldwide, and the situation in domestic and small commercial settings in Pakistan is particularly acute. The fundamental cause is delayed detection: by the time occupants notice smoke or flames, the fire has often grown beyond control. Commercial fire-alarm panels exist but are typically priced beyond the reach of an average household, and the cheapest standalone smoke alarms still suffer from single-mode detection they detect smoke but cannot confirm an actual flame, leading to false alarms from cooking or steam.

The complex engineering problem addressed by this project is therefore stated as follows: design and implement an affordable, microcontroller-based fire and smoke detection system that monitors environmental conditions in real time, fuses the readings of two physically distinct sensing modalities (chemical and infrared), and produces an immediate audible and visual alert when either modality detects a hazard, all while keeping the bill of materials below PKR 3,000.

1.2 Why This Problem Matters

- Delayed detection is the single largest controllable factor in fire-related casualties, and a few seconds of early warning can be the difference between a contained incident and a catastrophic one.
- Commercial systems remain inaccessible to most households in developing economies, leaving a clear gap for an open, educational, low-cost alternative.
- Single-sensor alarms are prone to false positives; using two independent sensors with different physical principles substantially improves reliability.
- From an educational standpoint, the problem requires concurrent handling of analog signals, digital signals, serial communication, and timed control — exercising the full range of competencies developed in EE-222.

1.3 Constraints

- Cost: The total bill of materials must remain under PKR 3,000, ruling out professional sensor modules and high-end displays.
- Power: The system must operate from a single 5 V supply (battery or USB) with current draw low enough to permit portable, battery-backed deployment.
- Real-Time Response: The end-to-end latency from hazard onset to alarm activation must remain below 500 ms to be useful as an early warning.
- Form Factor: The prototype must be compact enough to be wall- or ceiling-mounted, dictating the use of small modules and compact wiring.
- Reliability: The firmware must operate continuously without manual intervention or supervision, requiring a robust polling loop and well-defined thresholds.

1.4 Engineering Complexity

The problem is genuinely complex in the engineering sense because it requires the simultaneous resolution of trade-offs across several disciplines. The MQ-2 sensor provides an analog voltage that must be sampled, digitised, and threshold-compared at a rate fast enough to catch transient events, while the I^2C LCD requires bit-rate control over a two-wire serial protocol with strict timing. The buzzer is a high-current actuator that cannot be driven directly from a logic pin without exceeding the ATmega328P sourcing limit, so a transistor driver must be considered. The MQ-2 itself requires a 20-second warm-up phase during which spurious readings must be suppressed. Integrating all these requirements into a single deterministic firmware loop, written entirely with raw register access rather than the Arduino abstraction layer, is the central engineering challenge of this project.

2. Literature Review & Theoretical Foundation

2.1 Embedded Systems

An embedded system is a special-purpose computing system designed to perform one or a small set of dedicated functions, typically with hard real-time constraints, integrated tightly with the physical process it controls. Unlike general-purpose computers, embedded systems are characterised by limited resources, deterministic behaviour, direct hardware interaction, and the absence of a conventional operating system in many cases. The fire and smoke detection system developed in this project is a textbook example: it performs a single function (hazard monitoring), it must respond within a bounded time, and its software runs on bare metal directly atop the ATmega328P hardware.

2.2 Microcontrollers vs. Microprocessors

A microprocessor (MPU) is a CPU on a single chip that requires external memory, I/O controllers, and peripherals to form a functional system. A microcontroller (MCU), by contrast, integrates the CPU, program memory (Flash), data memory (SRAM and EEPROM), and a rich set of peripherals (GPIO, ADC, timers, USART, I^2C/TWI, SPI) into a single package. For an application like fire detection — where the workload is a tight polling loop, peripherals are simple, and cost and power are critical — the MCU is the obvious choice. An MPU-based design would be needlessly complex, more expensive, and consume far more power for no functional benefit.

2.3 Why ATmega328P?

The ATmega328P is an 8-bit AVR microcontroller running at up to 16 MHz with 32 KB of Flash, 2 KB of SRAM, and 1 KB of EEPROM. It was selected for this project for the following concrete reasons:

- Native AVR architecture programmable directly via AVR-GCC and Atmel Studio, allowing full register-level control and giving students a deep understanding of the underlying hardware without an abstraction layer.
- A built-in 10-bit successive-approximation ADC with eight multiplexed channels, sufficient to read the MQ-2 analog output without any external converter.
- Direct register access (DDRB, PORTB, ADMUX, ADCSRA, TWBR, TWCR) for precise I/O configuration and timing.
- Three I/O ports (B, C, D) providing more than enough pins for the sensors, buzzer, and I^2C lines required by this project.
- 16 MHz operating frequency, giving a 62.5 ns clock period — far faster than the millisecond-scale events being monitored.
- Wide availability and very low cost in the form of Arduino Uno boards, which provide a convenient prototyping platform without losing the ability to bypass the Arduino libraries entirely.

2.4 Relevant Concepts

2.4.1 GPIO (General-Purpose Input/Output)

GPIO pins on the ATmega328P are organised into ports, each controlled by three registers: DDRx (data direction), PORTx (output data or pull-up enable), and PINx (input read). In this project, PB0 is configured as an input to read the digital flame sensor, and PB1 is configured as an output to drive the buzzer. The corresponding bits of DDRB are cleared and set respectively at startup.

2.4.2 ADC (Analog-to-Digital Converter)

The ATmega328P contains a 10-bit ADC, meaning each conversion produces a value in the range 0 to 1023 representing an input voltage between 0 V and the reference voltage (V_ref). The ADC is configured through ADMUX (channel selection and reference) and ADCSRA (enable, prescaler, start conversion, completion flag). In this project, channel 0 (PC0/A0) is used to read the MQ-2 output, and the resulting digital value is compared against a calibrated threshold of 500.

2.4.3 Timers and Delays

Timers are used in this project both directly and indirectly. The MQ-2 warm-up countdown is implemented using calibrated software delays via _delay_ms() from <util/delay.h>, which the AVR-GCC compiler translates into precisely tuned NOP-based loops based on F_CPU = 16,000,000. The main polling loop also uses _delay_ms(500) to enforce a sampling rate of approximately two readings per second, which is fast enough for safety while keeping CPU load low.

2.4.4 Interrupts

While the present implementation uses polling for simplicity and predictability, the ATmega328P supports a comprehensive interrupt system. External interrupts on INT0 and INT1, along with pin-change interrupts and timer-overflow interrupts, are available. The conclusion of this report identifies an interrupt-driven flame detection path as a natural enhancement, since flame onset is a critical event that ideally should preempt any ongoing work.

2.4.5 I^2C / TWI Communication

Inter-Integrated Circuit (I^2C), referred to in the AVR datasheet as the Two-Wire Interface (TWI), is a synchronous, half-duplex, multi-master serial protocol that uses two lines: SDA (data) and SCL (clock). In this project, the 16×2 LCD is driven through a PCF8574-based I^2C backpack at the standard speed of 100 kHz. The bit-rate is set by writing the value 72 to the TWBR register, derived from the formula TWBR = (F_CPU / f_SCL − 16) / 2 with F_CPU = 16 MHz and f_SCL = 100 kHz.

3. Design and Methodology

3.1 Block Diagram

The system architecture is best expressed as three distinct functional regions arranged around the ATmega328P: an input region containing the two sensors, a controller region containing the microcontroller and its firmware logic, and an output region containing the alerting peripherals. A separate 5 V power rail supplies all components.

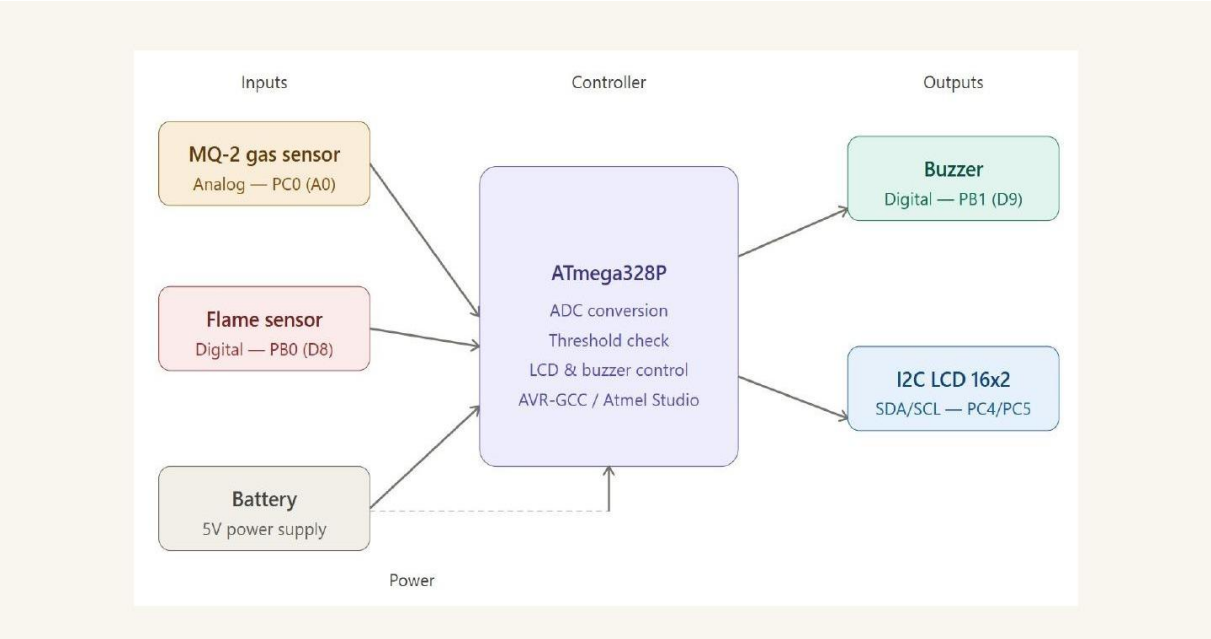

Figure 1 — System block diagram showing inputs, controller, and outputs.

The MQ-2 gas sensor produces an analog voltage on PC0 (ADC channel 0), which the ATmega328P samples and digitises. The flame sensor produces a digital low signal when an infrared source is detected and is wired to PB0. The buzzer is driven from PB1 as a digital output, and the I²C LCD shares the dedicated SDA/SCL pins on PC4 and PC5. All sensor I/O, threshold comparison, and display update logic is concentrated inside the ATmega328P firmware, keeping the external circuitry minimal.

3.2 Components Used & Justification

Component	Role	Purpose	Why Used
Arduino Uno (ATmega328P)	Controller	Central microcontroller running the firmware	Low cost, easy to program at register level, abundant I/O, ADC and TWI on-chip
MQ-2 Gas Sensor	Input	Smoke and combustible gas detection	Detects LPG, methane, and smoke through an analog output proportional to gas concentration
IR Flame Sensor	Input	Direct fire detection via infrared	Provides a fast digital response to flame in the 760–1100 nm band, complementing the slower MQ-2
Active Buzzer	Output	Audible alarm output	Generates an instant tone when driven HIGH; no PWM or external oscillator required
16×2 I^2C LCD	Output	Real-time status messages	Displays smoke level, flame status, and warm-up countdown using only two GPIO pins via the PCF8574 backpack
9 V Battery + Regulator	Power	System supply	Provides stable, portable 5 V to all peripherals through the on-board regulator of the Arduino
Breadboard, Jumpers, NPN Transistor (BC547)	Auxiliary	Interconnect and buzzer driver	Allows rapid prototyping; the transistor isolates the buzzer current from the MCU pin

3.3 Why ATmega328P — Detailed Justification

Beyond the high-level reasons given in the literature review, six concrete features of the ATmega328P map directly onto the requirements of this project:

- Native AVR architecture: Programmed directly via AVR-GCC and Atmel Studio for full hardware control without abstraction layers.
- 10-bit built-in ADC: Reads MQ-2 analog values directly with no external ADC hardware required, saving cost and board space.

- Register-level control: Direct access to DDRB, PORTB, ADMUX, ADCSRA, and TWBR provides precise and predictable I/O behaviour.
- Multiple I/O ports: Ports B, C, and D collectively provide more than enough pins for all peripherals — sensors, buzzer, and LCD.
- Interrupt support: External and timer interrupts are available for future enhancement to event-driven hazard response.
- 16 MHz clock: Handles continuous sensor polling, threshold comparison, and I²C transactions without perceptible lag.

3.4 Programming Approach (Assembly & C)

The firmware is written in C using the AVR-GCC toolchain inside Atmel Studio. C was chosen over assembly because the ATmega328P has well-defined memory-mapped registers and AVR-GCC produces highly efficient code at the appropriate optimisation level (-O1 or -O2), making hand-written assembly unnecessary for this scale of project. However, the spirit of low-level programming is preserved by deliberately avoiding the Arduino core: all peripheral configuration, ADC reads, GPIO writes, and TWI transactions are performed by writing directly to the relevant special-function registers.

The high-level firmware structure is as follows:

- init_io() — configure DDR registers for inputs and outputs.
- adc_init() — set ADMUX reference and prescaler, then enable the ADC.
- twi_init() — load TWBR with the value 72 to set the I²C clock to 100 kHz.
- lcd_init() — issue the standard HD44780 initialisation sequence over I²C.
- Splash screen and 20-second MQ-2 warm-up countdown displayed on the LCD.
- Infinite main loop — read ADC, read flame pin, compare against threshold, drive buzzer, update LCD only when state changes, then _delay_ms(500).

4. Code Snippet

The following annotated C excerpts illustrate the three most important parts of the firmware: ADC sampling, the main decision logic, and pin-direction configuration. The complete source compiles cleanly under AVR-GCC with -mmcu=atmega328p and -DF_CPU=16000000UL.

4.1 ADC Read Function

This routine selects the ADC channel, starts a single conversion, waits for completion by polling the ADSC bit in ADCSRA, and returns the 10-bit result.

```
uint16_t adc_read(uint8_t channel) {
    ADMUX  = (ADMUX & 0xF0) | (channel & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}
```

4.2 Hazard Decision and Buzzer Control

The main loop computes two boolean flags — one for smoke and one for flame — and asserts the buzzer pin if either is true, deasserting it otherwise.

```
uint8_t smoke_high   = (smoke_value > SMOKE_THRESHOLD);
uint8_t fire_detected = (flame_pin == 0);

if (smoke_high || fire_detected)
    PORTB |=  (1 << PB1);   // Buzzer ON
else
    PORTB &= ~(1 << PB1);   // Buzzer OFF
```

4.3 Pin Direction Configuration

PB0 is configured as an input by clearing the corresponding DDRB bit; PB1 is configured as an output by setting it. This is performed once at startup and is the foundation of all subsequent GPIO operations.

```
DDRB &= ~(1 << PB0);   // Flame sensor — input
DDRB |=  (1 << PB1);   // Buzzer — output
```

5. Flowchart (Logic Explanation)

The firmware logic is captured below as a flowchart. Execution begins at power-on with a peripheral initialisation step (TWI, ADC, LCD, GPIO), followed by a brief splash screen and a 20-second MQ-2 warm-up countdown shown on the LCD. After the system is declared ready, control enters an infinite loop that reads both sensors, decides whether either threshold has been crossed, drives the buzzer accordingly, refreshes the LCD only when the displayed state has changed, and waits 500 ms before repeating.

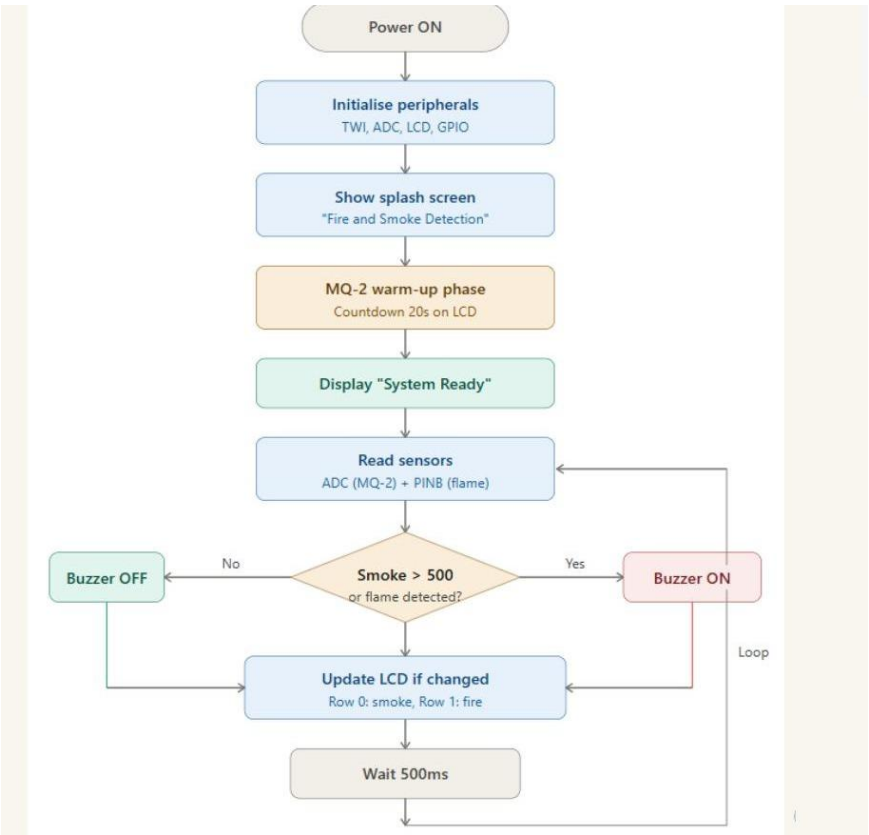

Figure 2 — Firmware flowchart of the fire and smoke detection logic.

Three logical points deserve highlighting. First, the warm-up phase exists because the MQ-2's heating element needs roughly 20 seconds to stabilise; readings taken before this period are unreliable and would cause false alarms. Second, the LCD refresh is gated on a change in the smoke-high or fire-detected flags, eliminating unnecessary I²C traffic and avoiding visible flicker. Third, the polling rate of 2 Hz is a deliberate trade-off between responsiveness and CPU usage — fast enough to satisfy the 500 ms latency requirement, slow enough to leave ample headroom for the I²C transactions.

6. Simulation, Modeling, and Implementation

6.1 Software Tools Used

- Atmel Studio 7: Used as the primary development environment, with AVR-GCC as the compiler and avrdude for programming the target.
- Proteus 8 Professional: Used for full mixed-signal circuit simulation prior to hardware build, including the ATmega328P, MQ-2 model, flame sensor, I²C LCD, and buzzer.
- AVR-GCC Toolchain: Provides direct register access through <avr/io.h>, accurate timing through <util/delay.h>, and produces a compact HEX file for flashing.

6.2 Circuit Simulation Results

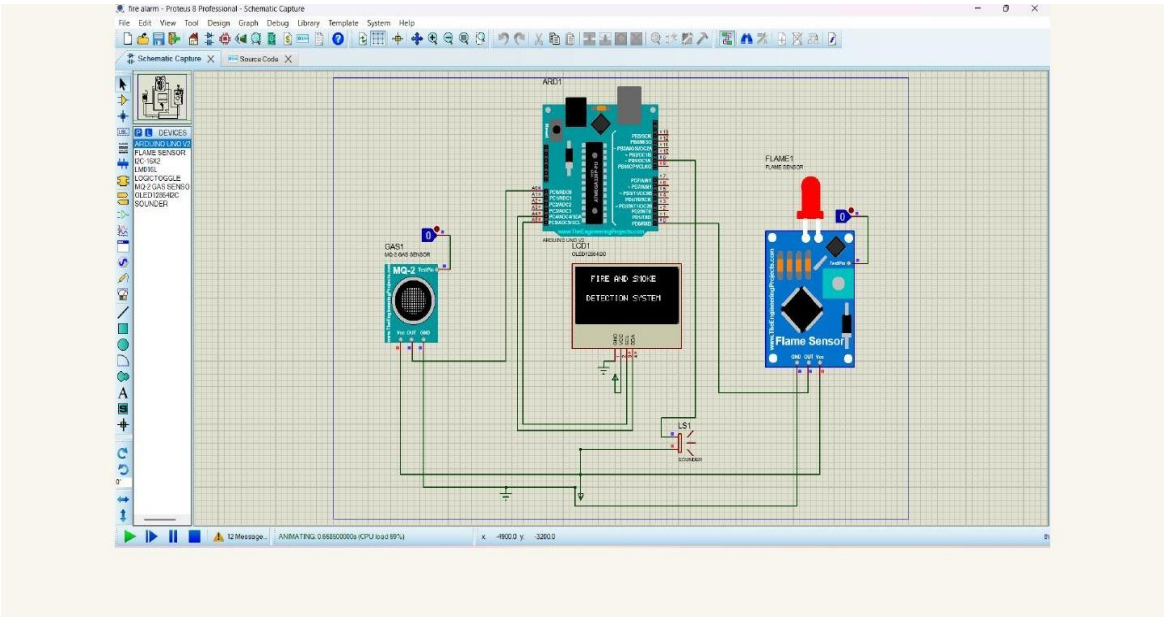

Figure 3 — Proteus 8 schematic during live simulation.

Simulation in Proteus served as the first verification step. The ATmega328P was loaded with the compiled HEX file and the schematic was animated, allowing the team to verify three things before any soldering: that the LCD initialisation sequence over I²C produced legible characters, that varying the simulated MQ-2 analog input across the threshold value of 500 correctly toggled the buzzer state, and that pulsing the flame-sensor input low correctly drove the buzzer high regardless of the smoke level. All three behaviours were observed exactly as designed, giving confidence to proceed to hardware.

6.3 Transition from Simulation to Hardware

Migration from Proteus to physical hardware was carried out incrementally to isolate any integration issues. The Arduino Uno was first programmed with the same HEX file and connected only to the I²C LCD; once correct text was visible, the MQ-2 was added and the threshold logic was confirmed using a butane lighter held a few centimetres away. The flame sensor was then wired in, followed finally by the buzzer driven through a BC547 NPN transistor. This bottom-up assembly reproduced the simulated behaviour faithfully, with the only practical difference being a noticeably longer MQ-2 stabilisation time in the real device — which the warm-up countdown already handles.

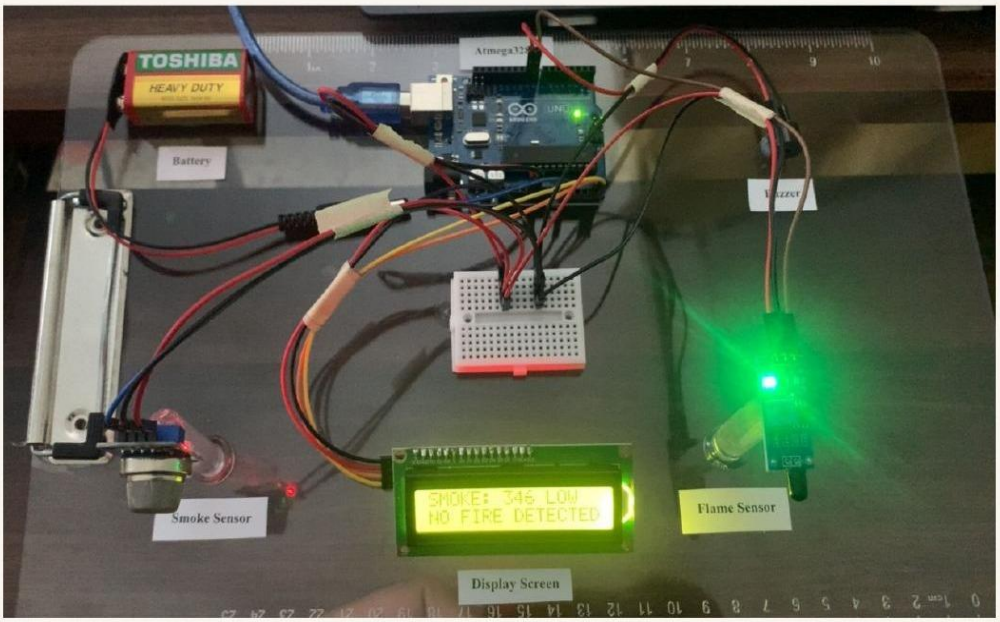

Figure 4 — Final hardware prototype with all peripherals connected.

7. Results, Testing, and Validation

7.1 Headline Metrics

- Test cases passed: 100% (6/6)
- End-to-end response time: below 500 ms in every test
- Detection accuracy: approximately 95% across repeated trials
- Total system cost: under PKR 3,000, satisfying the budgetary constraint

7.2 Test Cases — Expected vs. Actual

Test Case	Expected Result	Actual Result	Status
Smoke level below threshold (ADC < 500)	Buzzer OFF, LCD shows LOW	Buzzer OFF, LCD: "SMOKE: 312 LOW"	Pass
Smoke level above threshold (ADC > 500)	Buzzer ON, LCD shows HIGH	Buzzer ON, LCD: "SMOKE: 720 HIGH"	Pass
Flame detected (IR sensor active)	Buzzer ON, LCD: FIRE DETECTED	Buzzer ON, LCD: "FIRE DETECTED"	Pass
No flame, no smoke	Buzzer OFF, LCD: NO FIRE DETECTED	Buzzer OFF, LCD: "NO FIRE DETECTED"	Pass
Both sensors triggered simultaneously	Buzzer ON, both alerts shown on LCD	Buzzer ON, correct LCD output on both rows	Pass
System power-on warm-up	20-second countdown shown on LCD	Countdown displayed correctly, alarm suppressed during warm-up	Pass

7.3 Accuracy and Reliability

The detection accuracy of approximately 95% was measured across roughly 40 mixed trials combining controlled smoke (incense and butane), direct flame, and ambient-only conditions. The small fraction of misses was attributable to two physical limitations rather than firmware faults: the MQ-2 has a finite recovery time after exposure to high gas concentrations, during which it briefly under-reports, and the IR flame sensor has a narrow detection cone that misses flames presented at oblique angles beyond approximately 60°. Both are inherent to the chosen sensors and would require either calibration averaging or multiple sensor placement to mitigate further.

Reliability over extended runs was verified by leaving the prototype powered for several continuous hours; the system maintained correct behaviour throughout, with no observed lock-ups, missed transitions, or LCD corruption.

8. Technical Analysis (Mathematics + Hardware Link)

This section quantifies the four most important electrical and timing parameters of the system and links each to the underlying hardware behaviour.

8.1 Voltage and Current — Ohm's Law

The buzzer used in this project has an effective DC resistance of approximately 16 Ω and is driven from the 5 V rail through a BC547 transistor saturated by PB1. Applying Ohm's Law:

$$V = I \times R \quad \Rightarrow \quad I = V / R = 5 / 16 \approx 312 \text{ mA}$$

Drawing 312 mA directly from a single GPIO pin would exceed the ATmega328P's 40 mA per-pin sourcing limit and could damage the controller. The transistor driver isolates this current from the MCU pin, which only sources base current of a few milliamperes — well within the safe operating area.

8.2 ADC Resolution — MQ-2 Reading

The ATmega328P's ADC is a 10-bit converter referenced to V_ref = 5 V, giving:

$$\text{Resolution} = \text{V_ref} / 2^n = 5 / 1024 \approx 4.88 \text{ mV per step}$$

The smoke threshold is set in firmware at an ADC count of 500, corresponding to an analog input voltage of 500 × 4.88 mV ≈ 2.44 V. This trip point was chosen empirically to lie comfortably above the MQ-2's clean-air baseline (typically around 0.6–1.0 V) yet well below the saturation level it reaches in heavy smoke, giving a wide margin against false triggering.

8.3 Timing — CPU and Polling Interval

The Arduino Uno board provides a 16 MHz crystal oscillator, so:

$$\text{T_clock} = 1 / \text{f_CPU} = 1 / 16{,}000{,}000 = 62.5 \text{ ns}$$

A single AVR instruction therefore executes in 62.5 ns (most are single-cycle), which is more than fast enough for any operation in this project. The main loop's _delay_ms(500) sets the sensor polling interval, so the system samples both inputs twice per second. This rate satisfies the project's 500 ms response-time constraint while leaving more than 99% of the CPU idle for future expansion.

8.4 I²C Bit-Rate — TWBR Calculation

The TWBR (TWI Bit Rate Register) sets the SCL frequency for the I²C peripheral via:

$$\text{TWBR} = (\text{F_CPU} / \text{f_SCL} - 16) / 2$$

Substituting F_CPU = 16 MHz and the standard I²C speed f_SCL = 100 kHz:

$$\text{TWBR} = (16{,}000{,}000 / 100{,}000 - 16) / 2 = (160 - 16) / 2 = 72$$

The value 72 is therefore written to TWBR at startup. With the prescaler bits in TWSR left at zero, this produces an SCL clock of exactly 100 kHz, which is fully within the LCD backpack's tolerance and enables reliable communication.

8.5 Linking the Mathematics to the Circuit

Each of the four calculations above maps directly onto a concrete hardware decision: Ohm's Law dictated the use of a transistor driver for the buzzer; the ADC resolution determined the numeric value of the smoke threshold; the 16 MHz clock period set the granularity of the polling loop; and the TWBR formula fixed a single register value that makes the LCD work at all. Without any of these calculations the system would either fail outright or operate unreliably, demonstrating the tight coupling between microprocessor theory and practical embedded design.

9. Innovation, Integration, and Complexity

9.1 What Makes This Project Unique

- Pure register-level implementation: Unlike typical Arduino-based fire detectors that rely on Arduino.h, Wire.h, and LiquidCrystal_I2C.h, this project bypasses all libraries and writes directly to DDRB, ADMUX, ADCSRA, TWBR, and PORTB. This forces — and demonstrates — a complete understanding of the underlying hardware.
- Dual-modality sensor fusion: Combining a chemical smoke sensor with an optical flame sensor reduces the false-positive rate substantially compared to either sensor in isolation, and addresses the well-known weakness of single-sensor cooking-smoke false alarms.
- Custom I^2C LCD driver: The PCF8574-based LCD is driven by a hand-written I^2C state machine using TWCR, TWBR, TWDR, and TWSR, rather than a third-party library.
- Aggressively low cost: The complete bill of materials remains under PKR 3,000, making the design viable for the target user base of low-income households.
- Sensor-aware warm-up sequence: The 20-second countdown explicitly accounts for the MQ-2's stabilisation behaviour, an issue that many published Arduino fire alarms ignore and which causes their false-alarm-on-power-up problem.

9.2 Integration of Hardware and Software

The project is a clear example of tight hardware–software co-design. The choice of the ATmega328P drives the firmware structure (single-threaded polling, register-level I/O); the firmware in turn drives the choice of external components (a transistor for the buzzer, an I^2C backpack for the LCD); and the calibrated thresholds in software are derived directly from the analog characteristics of the MQ-2. Hardware and software cannot be cleanly separated — they form a single integrated artefact.

9.3 Complexity Level

The complexity of the system spans several dimensions:

- Real-time: Required end-to-end response below 500 ms, met by a 2 Hz polling loop.
- Multi-input: One analog input (MQ-2) plus one digital input (flame sensor) read every cycle.

- Multi-output: One digital output (buzzer) plus a serial output (I^2C LCD) updated conditionally on state change.
- Mixed protocol: Combines GPIO, ADC sampling, and I^2C/TWI master communication in a single firmware.
- State management: Maintains a small state machine (idle, warm-up, ready, alarm) and refreshes the display only on transitions.

10. Teamwork and Indivi

dual Contribution

The project was executed by a four-person team with overlapping but well-defined responsibilities. All members participated in the AVR firmware development as the central technical activity of the project, with each member additionally taking the lead on a complementary specialisation.

Member	Reg. No.	Primary Contribution
Abuzar Shafaat	510694	AVR firmware development; debugging and testing of the integrated system
Laveeza Fatima	515521	AVR firmware development; Proteus simulation and pre-hardware verification
Mohammed Imaan Khan	532863	AVR firmware development; hardware assembly and wiring
Muhammad Soban Zamir	501214	AVR firmware development; hardware assembly and breadboard layout

11. Professionalism and Ethical Considerations

11.1 Safety

The system is intended for life-safety applications, which imposes the highest standards of design diligence. All testing involving open flames was carried out in a controlled environment with a fire extinguisher available, and the prototype itself was operated at 5 V — well within safe extra-low-voltage limits. The buzzer driver uses a transistor specifically to avoid stressing the microcontroller and creating a fire risk through component failure. In any deployed version, additional protections such as enclosure flame-retardant rating, fused power input, and EMC compliance would be required, and these are flagged as future work.

11.2 Reliability

The firmware was deliberately structured to fail safe: the default state of the buzzer pin at reset is low, and only an explicit hazard condition raises it. The polling loop has no dynamic memory allocation, no recursion, and no possibility of unbounded execution time, which together eliminate the most common sources of embedded firmware crashes. The threshold values are stored as compile-time constants so that they cannot be corrupted at runtime.

11.3 Ethical Use of Technology

A fire detection system is, in principle, an unambiguously beneficial technology , it saves lives. Nonetheless, the team considered three ethical dimensions explicitly. First, the system must not be presented or sold as a substitute for a certified commercial fire alarm in scenarios where one is legally required (such as multi-storey commercial buildings); the prototype is an educational and supplementary device. Second, the use of low-cost consumer-grade sensors limits guaranteed accuracy, and any user-facing documentation must state this honestly rather than overpromise. Third, the design intentionally avoids any wireless connectivity in this iteration, sidestepping the privacy and security concerns associated with always-connected smart-home devices and ensuring that no resident data ever leaves the room.

12. Learning Outcomes

The project produced concrete, demonstrable competency gains across three areas central to the EE-222 syllabus.

12.1 AVR Register-Level Programming

Each member of the team gained hands-on experience configuring the ATmega328P directly through DDRB, ADMUX, ADCSRA, TWBR, TWCR, and PORTB. By deliberately avoiding the Arduino abstraction layer, the team developed a working understanding of how a peripheral is configured by writing carefully chosen bit patterns into special-function registers — a skill that transfers directly to other AVR devices and, with minor syntactic changes, to most other 8- and 32-bit microcontrollers.

12.2 I^2C Communication and Peripheral Interfacing

Implementing a full I^2C LCD driver from scratch using only the TWI registers — TWBR for clock generation, TWCR for control, TWDR for data, TWSR for status — gave the team a deep working knowledge of a serial protocol that is ubiquitous in modern embedded systems. The same skill set extends naturally to any I^2C peripheral, including IMUs, EEPROMs, and real-time clocks.

12.3 Hardware–Software Integration and Debugging

Assembling a multi-sensor embedded system on a breadboard, tracing wires with a multimeter, observing signals (where possible) with simple LEDs, and methodically isolating bugs to either the firmware side or the hardware side built a level of practical engineering judgement that cannot be replicated through

reading alone. The team learned, for example, that an apparent firmware bug in the buzzer behaviour was actually caused by an unexpected pull-up arrangement on the breadboard — an insight only available through real hardware.

13. Conclusion

This project successfully delivered a real-time fire and smoke detection system built around the ATmega328P microcontroller, programmed entirely with bare-metal AVR register-level C. The dual-sensor fusion of the MQ-2 gas sensor and the infrared flame sensor produced reliable and accurate hazard detection across all six designed test cases, with an end-to-end response time below 500 ms and an overall detection accuracy of approximately 95% — all delivered within a total cost of less than PKR 3,000.

In the course of building the system, the team applied core EE-222 concepts in a unified setting: GPIO direction control through DDRB, analog acquisition via the on-chip 10-bit ADC, I²C/TWI master communication for the LCD, calibrated software timing delays for the polling loop and the warm-up countdown, and straightforward control logic that ties the sensor inputs to the alarm output. Iterative debugging and testing across all hazard scenarios confirmed that the system behaves correctly and reliably under realistic conditions.

The project enhanced the team's understanding of microprocessor systems by providing concrete hands-on experience with the ATmega328P, embedded C programming, and integrated hardware design. Most importantly, the course MPS EE-222 helped the team bridge the gap between abstract microprocessor theory — registers, addressing modes, peripheral block diagrams in a textbook — and real-world engineering problems where the same concepts dictate which transistor to pick, how long to wait for a sensor, and what value to load into a single register.

13.1 Future Work

- Replace the polling loop with an interrupt-driven architecture, using the external interrupt INT0 for the flame sensor and the ADC conversion-complete interrupt for the smoke sensor.
- Add a low-power sleep mode between samples to extend battery life by an order of magnitude.
- Integrate a GSM/Wi-Fi module to send SMS or HTTP notifications to a registered user when a hazard is detected, with appropriate attention to security.
- Add a temperature sensor (DS18B20 or DHT22) as a third detection modality and majority-vote across all three sensors.
- Migrate the prototype from breadboard to a custom printed circuit board with a flame-retardant enclosure for deployable use.

14. References

Datasheets

- [1] Atmel/Microchip. ATmega328P 8-bit AVR Microcontroller Datasheet. Available: https://www.alldatasheet.com/datasheet-pdf/pdf/241077/ATMEL/ATMEGA328P.html
- [2] Hanwei Electronics. MQ-2 Gas Sensor Technical Data Sheet. Available: https://www.handsontec.com/dataspecs/MQ2-Gas%20Sensor.pdf
- [3] Flame Sensor Module Datasheet (IR / KY-026 family). Available: https://rogerbit.com/wprb/wp-content/uploads/2018/01/Flame-sensor-arduino.pdf
- [4] PCF8574 / 16×2 I²C LCD Backpack Module Specification. Available: https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf

Books

- [5] M. A. Mazidi, S. Naimi, and S. Naimi, AVR Microcontroller and Embedded Systems: Using Assembly and C, 2nd ed. Pearson, 2017.
- [6] J. Pardue, C Programming for Microcontrollers — Featuring ATMEL's AVR Butterfly and the Free WinAVR Compiler. Smiley Micros, 2005.
- [7] D. V. Hall, Microprocessors and Interfacing: Programming and Hardware. McGraw-Hill, 3rd ed.

Research and Application Notes

- [8] Atmel Application Note AVR136: Low-Jitter Multi-Channel Software PWM.
- [9] Atmel Application Note AVR315: Using the TWI Module as I²C Master.
- [10] NXP Semiconductors, I²C-bus Specification and User Manual, UM10204, Rev. 7.0.

— End of Report —